Module 8

JavaScript

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

JavaScript is a scripting language used to create interactive and dynamic content on websites. It is one of the core technologies of web development, along with HTML and CSS.

It gives **life and interactivity** to websites.

Without JavaScript → websites are just **static pages**.

Role of JavaScript in Web Development

In web development, we usually talk about three main parts:

1. **HTML** → Structure of the page (like bones).
2. **CSS** → Styling/Design (like skin & clothes).
3. **JavaScript** → Behavior/Actions (like brain & movement).

Question 2: How is JavaScript different from other programming languages like Python or Java?

1. Execution Environment

- JavaScript → Runs mainly in web browsers (front-end), though with Node.js it can also run on servers.

- Python / Java → Usually run on servers, desktops, or command line. Not built into browsers.

2. Typing System

- JavaScript → *Loosely typed / dynamically typed* (no need to declare variable types). Example:
- let x = 5; // number
- x = "hello"; // valid in JS
- Java → *Strongly typed & statically typed* (must declare variable type).
- Python → Dynamically typed, but stricter than JS in some cases.

3. Compilation vs. Interpretation

- JavaScript → Interpreted (executed directly by browser/engine like Chrome's V8).
- Java → Compiled into *bytecode*, runs on the JVM (Java Virtual Machine).
- Python → Interpreted (runs line by line using Python interpreter).

4. Use Cases

- JavaScript → Mainly for web development (front-end + back-end with Node.js).
- Python → Used in data science, AI/ML, scripting, automation, back-end web dev.
- Java → Popular in enterprise apps, Android apps, banking systems.

5. Syntax

- JavaScript syntax is similar to C.

- Python emphasizes readability (indentation instead of {} brackets).
- Java is verbose, requires strict structure (e.g., class-based).

5. Performance

JavaScript → Fast in browsers (because of V8 engine), but not as fast as Java for heavy computing.

Python → Slower, not great for CPU-heavy tasks (but excellent for prototyping, AI, etc.).

Java → Very fast and optimized, great for enterprise applications.

---

6. Object-Oriented Approach

JavaScript → Prototype-based OOP.

Python → Class-based OOP (more traditional).

Java → Purely class-based OOP.

Question 3: Discuss the use of <script> tag in HTML. How can you link an external JavaScript file to an HTML document?

The <script> tag in HTML is used to **run JavaScript** on a webpage.

You can use it in **two ways**:

1. Write JavaScript **inside the HTML file**.
2. Link to an **external JavaScript file (.js)**.

- <script> = tells browser to **run JavaScript**.
-
    - **Inside HTML** → small projects.
    - **External file (.js)** → big projects (cleaner and reusable).

1. Internal Script (inside HTML)

You put JavaScript **directly inside <script>**:

```
<!DOCTYPE html>
<html>
<head>
 <title>Internal Script</title>
 <script>
  function greet() {
   alert("Hello!");
  }
 </script>
</head>
<body>
 <button onclick="greet()">Click Me</button>
</body>
</html>
```

2. External Script (separate file)

You keep JavaScript in a **separate file** and link it:

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>External Script</title>
  <!-- Link external JS file -->
  <script src="script.js"></script>
</head>
<body>
  <button onclick="greet()">Click Me</button>
</body>
</html>
```

script.js

```
function greet() {
  alert("Hello from external file!");
}
```

Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

- **Variables in js:-**

A variable is like a container (box) used to store data.

Example: You can store a number, a name, or any value in a variable**.**

var → old, can re-declare & update.

let → modern, block-scoped, can update but not re-declare.

const → block-scoped, fixed values, cannot change.

Keyword	Scope	Re-declare	Update	Best Use
var	Function scope	Yes	Yes	Old code / legacy
let	Block scope	No	Yes	When value changes
const	Block scope	No	No	Fixed values

Question 2: Explain the different data types in JavaScript. Provide examples for each.

Data Types in JavaScript

JavaScript has two main categories of data types:

1. **Primitive (basic types):- Primitive = single/simple values.**
2. **Non-Primitive (objects, collections, etc.):- Non-Primitive = collections/more complex.**

1. **Primitive Data Types (Basic / Simple)**

   These hold **single values**.
   Examples:

- **Number** → let age = 21;
- **String** → let name = "Tisha";
- **Boolean** → let isStudent = true;
- **Undefined** → let x;
- **Null** → let y = null;
- **Symbol** → let id = Symbol("abc");
- **BigInt** → let big = 1234567890123456789 0n;

2. Non-Primitive Data Types (Complex / Collection)

These hold **multiple values or objects**.
Examples:

- **Object** → let person = {name: "Tisha", age: 21};
- **Array** → let fruits = ["Apple", "Banana"];
- **Function** → function greet(){ return "Hi"; }
- • **Question 3: What is the difference between undefined and null in JavaScript?**
- • **undefined** → A variable is declared but not given any value yet. (JavaScript sets it automatically.)
-
- • **null** → A special value that means "nothing" or "empty," set by the programmer.

Point	undefined	null
Meaning	Variable is declared but no value assigned	Value is manually set to empty
Type	undefined (its own type)	object (special object value)
Set By	JavaScript automatically	Programmer intentionally
Example	let a; console.log(a); // undefined	let b = null; console.log(b); // null

JavaScript Operators

Question 1: What are the different types of operators in JavaScript? Explain with examples.

1 Arithmetic Operators → For math work

- + (add), - (subtract), * (multiply), / (divide), % (modulus), ** (power), ++ (increment), -- (decrement)

2 Assignment Operators → To assign values

- = , +=, -=, *=, /=, %=

3 Comparison Operators → To compare values (true/false)

- ==, ===, !=, !==, >, <, >=, <=

4 Logical Operators → To combine conditions

- && (AND), || (OR), ! (NOT)

Question 2: What is the difference between == and === in JavaScript?

Point	== (Double Equals)	=== (Triple Equals)
Name	Loose Equality	Strict Equality
Checks	Only **value**	**Value + Type**
Type Conversion	Yes (converts if types differ)	No (must be same type)
Example	5 == "5" → true	5 === "5" → false
Safe to Use?	Not always (can give confusing results)	Recommended (clear and strict)

Control Flow (If-Else, Switch)

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

What is Control Flow in JavaScript?

• Control flow is the order in which statements are executed in a program.

• Normally, code runs top to bottom, line by line.

• With control flow statements (like if-else, switch, loops), we can change the

execution path depending on conditions.

if-else Statement in JavaScript

The if-else statement is used to make decisions in code.

Syntax:14 | P a g e

```
if (condition) {
// code runs if condition is true
} else {
// code runs if condition is false
}
```

Example:

```
let age = 18;
if (age >= 18) {
console.log("You are eligible to vote.");
} else {
console.log("You are not eligible to vote.");
}
```

How it works:

• If age >= 18 is true, it prints "You are eligible to vote."

• Otherwise, it prints "You are not eligible to vote."

Key Points:

• if checks the condition.

• else runs only if the if condition is false.

• You can also use else if for multiple conditions.

Example:

```
let marks = 75;

if (marks >= 90) {

console.log("Grade A");

} else if (marks >= 60) {

console.log("Grade B");15 | P a g e

} else {

console.log("Grade C");

}
```

In short:

• Control flow decides the order of execution.

• if-else allows the program to take different paths based on conditions.

Question 2: Describe how switch statements work in JavaScript.

When should you use a switch statement instead of if-else?

Switch Statement in JavaScript

• A switch statement is used to execute one block of code out of many options.

• It’s often used as a cleaner alternative to multiple if-else-if statements.

Syntax:

```
switch (expression) {

case value1:

// code runs if expression === value1

break;

case value2:
```

```
// code runs if expression === value2
break;
default:
// code runs if no case matches
}
```

• expression is evaluated once.

• The result is compared with each case value using strict comparison (===).16 | P a g e

• break ends the switch (without it, execution “falls through” to the next case).

• default runs if no cases match.

Example:

```
let day = 3;
switch (day) {
case 1:
console.log("Monday");
break;
case 2:
console.log("Tuesday");
break;
case 3:
console.log("Wednesday");
break;
case 4:
```

```
console.log("Thursday");
break;
default:
console.log("Invalid day");
}
```

Output: Wednesday

When to Use switch Instead of if-else?

• Use if-else when you need to check complex conditions (e.g., ranges, logical

operators).

• Use switch when you need to compare one value against multiple possible exact

matches.

Example use cases for switch:

• Menu selection

• Day of the week

• User role (admin, editor, viewer)

• Handling multiple fixed options

In short:

• switch is best for multiple exact matches.

• if-else is best for conditions with ranges or complex logic.

Loops (For, While, Do-While)

Question 1: Explain the different types of loops in JavaScript (for,

while, do-while). Provide abasic example of each.

Loops in JavaScript

A loop is used to execute a block of code repeatedly as long as a condition is true.

The main types of loops in JavaScript are:

1. for loop

2. while loop

3. do-while loop

1. for loop

• Best when the number of iterations is known in advance.

Syntax:

```
for (initialization; condition; update) {

// code to be executed

}
```

Example:

```
for (let i = 1; i <= 5; i++) {18 | P a g e

console.log("Number: " + i);

}
```

Prints numbers 1 to 5.

2. while loop

• Best when the number of iterations is not known and depends on a condition.

Syntax:

```
while (condition) {

// code to be executed
```

}

Example:

```
let i = 1;
while (i <= 5) {
console.log("Number: " + i);
i++;
}
```

Prints numbers 1 to 5.

3. do-while loop

• Similar to while, but it executes at least once (because the condition is checked after

execution).

Syntax:

```
do {
// code to be executed
} while (condition);
```

Example:

```
let i = 1;
do {
console.log("Number: " + i);
i++;19 | P a g e
} while (i <= 5);
```

Prints numbers 1 to 5 (runs at least once even if condition is false).

Summary Table

Loop

Type

Condition

Checked

Executes At Least

Once?

Best Used For

for

Before each loop

No

When number of iterations is

known

while

Before each loop

No

When condition must be true to

start

do-while

After each loop

Yes

When code should run at least

once

In short:

• for → fixed repetitions.

• while → repeats while condition is true.

• do-while → executes at least once, then checks condition.

Question 2: What is the difference between a while loop and a do while loop?

Difference Between while loop and do-while loop

1. Condition Checking

• while loop: Condition is checked before executing the loop body.

• do-while loop: Condition is checked after executing the loop body.

2. Minimum Execution

• while loop: May run 0 times if condition is false initially.

• do-while loop: Runs at least once, even if condition is false.

3. Syntax & Example20 | P a g e

while loop:

```
let i = 5;
while (i < 5) {
console.log("Hello");
i++;
}
```

Output: nothing (condition false at start).

do-while loop:

```
let i = 5;
do {
console.log("Hello");
i++;
```

} while (i < 5);

Output: Hello (runs once before checking condition).

Summary Table

Feature	while loop	do-while loop
Condition checked	Before body	After body
Minimum executions	0 times	1 time
Use case	When loop should only run if condition is true	When loop must run at least once

In short:

• while → checks first, may not run.

• do-while → runs once, then checks.21 | P a g e

Functions

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

What are Functions in JavaScript?

A function in JavaScript is a block of code designed to perform a specific task.

• It helps to reuse code (write once, use many times).

• Improves readability and maintainability of programs.

Syntax for Declaring a Function

```
function functionName(parameters) {
// block of code
return value; // optional
}
```

• function → keyword to declare a function.

• functionName → name you give to the function.

• parameters → values you pass into the function (optional).

• return → sends a value back (optional).

Syntax for Calling a Function

```
functionName(arguments);
```

• arguments → actual values passed to the function.

Example

```
// Function declaration
function greet(name) {
console.log("Hello, " + name + "!");
}22 | P a g e
// Function call
greet("Charmi");
greet("Alex");
```

Output:

Hello, Charmi!

Hello, Alex!

Key Points

• A function runs only when it is called.

• You can call it multiple times with different values.

• Functions can return values or just perform an action.

In short:

Functions = reusable blocks of code.

Declare → function myFunc() { ... }

Call → myFunc();

Question 2: What is the difference between a function declaration and a function expression?

1. Function Declaration

A function declaration defines a named function using the function keyword.

• It is hoisted (can be called before it is defined).

Syntax:

```
function greet() {
console.log("Hello!");
}
greet(); // Works even if called before declaration
```

2. Function Expression23 | P a g e

A function expression creates a function and assigns it to a variable.

• It is not hoisted (can only be called after it is defined).

Syntax:

```
const greet = function() {
console.log("Hello!");
};
greet(); // Works only if called after the definition
```

Key Differences

Feature	Function Declaration	Function Expression
Hoisting	Yes (can call before definition)	No (must define first)
Name	Always has a name	Can be named or anonymous
When to use	For general reusable functions	When assigning to variables, passing as arguments, or using callbacks

Example Showing the Difference

```
// Function Declaration
sayHello(); // Works (hoisted)
function sayHello() {
console.log("Hello from declaration!");
}
// Function Expression
sayHi(); // Error (not hoisted)
const sayHi = function() {
console.log("Hello from expression!");
};24 | P a g e
```

In short:

• Function Declaration → hoisted, can be called before it's defined.

• Function Expression → not hoisted, must be defined before calling.

Question 3: Discuss the concept of parameters and return values in functions.

Parameters in Functions

• Parameters are variables listed inside the function's parentheses.

• They act as placeholders for values that will be passed when the function is called.

Example with Parameters:

```
function greet(name) { // 'name' is a parameter
console.log("Hello, " + name + "!");
}
greet("Charmi"); // "Charmi" is an argument
```

greet("Alex");

Output:

Hello, Charmi!

Hello, Alex!

Return Values in Functions

• A function can return a value using the return keyword.

• The returned value can be stored in a variable or used directly.

Example with Return Value:

function add(a, b) {

return a + b; // returns the sum

}25 | P a g e

let result = add(5, 3);

console.log(result); // 8

Here:

• a and b are parameters.

• return a + b; sends back the result.

• result stores the returned value.

Key Points

• Parameters = input placeholders inside the function.

• Arguments = actual values passed when calling.

• Return value = output given back by the function.

In short:

Parameters allow functions to take inputs, and return allows functions to give outputs.

Arrays

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

An array is a special type of object used to store multiple values in a single variable.

• Values in an array are called elements.

• Each element has an index (starting from 0).

• Arrays can hold any data type: numbers, strings, objects, even other arrays.

Declaring and Initializing Arrays

1. Using Square Brackets [] (most common)

let fruits = ["Apple", "Banana", "Mango"];26 | P a g e

console.log(fruits); // ["Apple", "Banana", "Mango"]

console.log(fruits[0]); // Apple (first element)

2. Using new Array() Constructor

let numbers = new Array(10, 20, 30, 40);

console.log(numbers); // [10, 20, 30, 40]

Note: Using new Array(5) creates an array with 5 empty slots, not numbers.

3. Empty Array then Assigning Values

let colors = [];

colors[0] = "Red";

colors[1] = "Green";

colors[2] = "Blue";

console.log(colors); // ["Red", "Green", "Blue"]

Example with Mixed Data Types

let mixed = [100, "Hello", true, { name: "Charmi" }, [1, 2, 3]];

console.log(mixed);

Arrays can hold different types of values together.

Key Points

• Arrays are zero-indexed (first element at index 0).

• Can store multiple values in one variable.

• Declared using [] (preferred) or new Array().

In short:

An array = a collection of values in a single variable.

Example: let arr = [1, 2, 3];

An array is a special type of object used to store multiple values in a single variable.27 | P a g e

• Values in an array are called elements.

• Each element has an index (starting from 0).

• Arrays can hold any data type: numbers, strings, objects, even other arrays.

Declaring and Initializing Arrays

1. Using Square Brackets [] (most common)

let fruits = ["Apple", "Banana", "Mango"];

console.log(fruits); // ["Apple", "Banana", "Mango"]

console.log(fruits[0]); // Apple (first element)

2. Using new Array() Constructor

let numbers = new Array(10, 20, 30, 40);

console.log(numbers); // [10, 20, 30, 40]

Note: Using new Array(5) creates an array with 5 empty slots, not numbers.

3. Empty Array then Assigning Values

let colors = [];

colors[0] = "Red";

colors[1] = "Green";

colors[2] = "Blue";

console.log(colors); // ["Red", "Green", "Blue"]

Example with Mixed Data Types

let mixed = [100, "Hello", true, { name: "Charmi" }, [1, 2, 3]];

console.log(mixed);

Arrays can hold different types of values together.

Key Points28 | P a g e

• Arrays are zero-indexed (first element at index 0).

• Can store multiple values in one variable.

• Declared using [] (preferred) or new Array().

In short:

An array = a collection of values in a single variable.

Example: let arr = [1, 2, 3];

Question 2: Explain the methods push(), pop(), shift(), and unshift() used in arrays.

1. push()

• Adds one or more elements to the end of the array.

• Returns the new length of the array.

Example:

```
let fruits = ["Apple", "Banana"];
fruits.push("Mango");
console.log(fruits); // ["Apple", "Banana", "Mango"]
```

2. pop()

• Removes the last element from the array.

• Returns the removed element.

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
let removed = fruits.pop();
console.log(fruits); // ["Apple", "Banana"]
console.log(removed); // "Mango"
```

3. shift()

• Removes the first element from the array.

• Returns the removed element.29 | P a g e

Example:

```
let fruits = ["Apple", "Banana", "Mango"];
let removed = fruits.shift();
console.log(fruits); // ["Banana", "Mango"]
console.log(removed); // "Apple"
```

4. unshift()

• Adds one or more elements to the beginning of the array.

• Returns the new length of the array.

Example:

let fruits = ["Banana", "Mango"];

fruits.unshift("Apple");

console.log(fruits); // ["Apple", "Banana", "Mango"]

Summary Table

Method Action

Returns

push()

Adds element(s) to the end

New array length

pop()

Removes last element

Removed element

shift()

Removes first element

Removed element

unshift() Adds element(s) to beginning New array length

In short:

- **push() → add to end**
- **pop() → remove from end**
- **shift() → remove from start**
- **unshift() → add to start**

Question 1: What is an object in JavaScript? How are objects

different from arrays?

• An object is a collection of key-value pairs (also called properties).

• Keys (also called properties) are strings, and values can be any data type including

arrays, functions, or other objects.

• Objects are used to represent real-world entities and store structured data.

Example:

```
let person = {

name: "Charmi",

age: 23,

isStudent: true

};

console.log(person.name); // "Charmi"

console.log(person.age); // 23
```

Difference Between Objects and Arrays

Feature	Object	Array
Structure	Key-value pairs	Ordered list of values
Access	By key (obj.key or obj["key"])	By index (arr[0])

Use Case Represent entities with properties Store collections of items

Example

{name: "Alice", age: 20}

[10, 20, 30, 40]

Key Points

• Objects = unordered named properties.

• Arrays = ordered indexed elements.31 | P a g e

• Arrays are a type of object in JavaScript, but with numeric indexes and special

methods like push, pop.

In short:

• Objects = store data as properties (key-value pairs).

• Arrays = store data as lists (ordered by index).

Question 2: Explain how to access and update object properties

using dot notation and bracket notation.

Accessing and Updating Object Properties in JavaScript

There are two main ways to access and update object properties:

1. Dot Notation

2. Bracket Notation

1. Dot Notation

• Use the dot (.) followed by the property name.

• Works only when the property name is a valid identifier (no spaces, doesn’t start

with a number).

Example:

```
let person = {
name: "Charmi",
age: 23
};
// Access property
console.log(person.name); // "Charmi"
// Update property
person.age = 26;
console.log(person.age); // 2632 | P a g e
// Add new property
person.city = "Ahmedabad";
console.log(person.city); // "Ahmedabad"
```

2. Bracket Notation

• Use square brackets [ ] with the property name as a string.

• Useful when property names have spaces, special characters, or dynamic keys.

Example:

```
let person = {
"first name": "Charmi",
age: 25
};
// Access property
console.log(person["first name"]); // "Charmi"
```

```
// Update property
person["age"] = 26;
console.log(person["age"]); // 26
// Add new property dynamically
let key = "city";
person[key] = "Ahmedabad";
console.log(person.city); // "Ahmedabad"
```

Key Points

Feature	Dot Notation	Bracket Notation
Syntax	obj.key	obj["key"]33 | P a g e
When to use	Normal property names	Special characters, spaces, or dynamic keys
Access & Update	obj.key = value	obj["key"] = value

In short:

• Dot notation → simple and common.

• Bracket notation → flexible for dynamic or special property names.

JavaScript Events

Question 1: What are JavaScript events? Explain the role of event listeners.

• Events are actions or occurrences that happen in the browser, usually triggered by

the user or the browser itself.

• Examples of events:

o User clicks a button → click

o User moves the mouse → mousemove

o User types in an input field → keydown, keyup

o Page finishes loading → load

• Events allow web pages to be interactive.

Role of Event Listeners

• An event listener is a function that waits for a specific event to occur on an element

and then executes code.

• It “listens” for the event and responds when it happens.

Syntax:

element.addEventListener(event, function, useCapture);

• event → the type of event to listen for (click, mouseover, etc.)

• function → the function to run when the event occurs

• useCapture → optional, usually false34 | P a g e

Example:

<button id="myBtn">Click Me</button>

<script>

let button = document.getElementById("myBtn");

// Add event listener

```
button.addEventListener("click", function() {
alert("Button was clicked!");
});
</script>
```

How it works:

• The browser listens for a click event on the button.

• When clicked, the function runs and shows the alert.

Key Points:

• Events = actions happening in the browser.

• Event listeners = functions that respond to events.

• Using addEventListener is preferred over inline HTML events (onclick) because it

keeps JavaScript separate from HTML.

In short:

• Events = triggers (click, hover, type).

• Event listeners = “watchers” that execute code when events happen.

Question 2: How does the addEventListener() method work in

JavaScript? Provide an example.

How addEventListener() Works in JavaScript35 | P a g e

• The addEventListener() method attaches an event handler to an HTML element.

• It allows the element to “listen” for a specific event and execute a function when

that event occurs.

• Advantages over inline events:

o You can attach multiple events to the same element.

o Keeps JavaScript separate from HTML.

Syntax:

element.addEventListener(event, function, useCapture);

Parameter

Description

event

Type of event (e.g., "click", "mouseover")

function

Function to execute when the event occurs

useCapture Optional, true or false (usually false)

Example:

```
<button id="myBtn">Click Me</button>
<script>
let button = document.getElementById("myBtn");
// Attach a click event listener
button.addEventListener("click", function() {
alert("Button clicked!");
});
</script>
```

Explanation:

1. getElementById("myBtn") selects the button element.

2. addEventListener("click", function(){...}) attaches a click event to it.

3. When the button is clicked, the function runs and shows an alert.

Key Points:

• addEventListener allows dynamic and multiple event handling.

• Event handler can be an anonymous function or a named function.

Example with a named function:

```
function showMessage() {
alert("Hello!");
}
button.addEventListener("click", showMessage);
```

In short:

• addEventListener() = a way to listen and respond to events on HTML elements.

• Preferred over inline events for cleaner, flexible code.

DOM Manipulation

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

• DOM (Document Object Model) is a programmatic representation of an HTML

document.

• It represents the page as a tree structure where:

o Nodes = elements, text, attributes

o Branches = relationships between elements (parent, child, siblings)

• The DOM allows JavaScript to access, modify, and manipulate HTML elements

dynamically.

Example of DOM Structure:37 | P a g e

For this HTML:

<!DOCTYPE html>

<html>

<body>

<h1 id="title">Hello World</h1>

Welcome to JavaScript!

</body>

</html>

The DOM tree looks like:

document

└─ html

└─ body

├─ h1#title

└─ p

How JavaScript Interacts with the DOM

• Access Elements: Find HTML elements using methods like:

o getElementById(), getElementsByClassName(), querySelector(),

querySelectorAll()

• Modify Content: Change text or HTML inside elements using innerText or

innerHTML.

• Change Styles: Modify CSS using style property.

• Add/Remove Elements: Use methods like appendChild(), removeChild().

• Handle Events: Attach event listeners using addEventListener().

Example: Interacting with the DOM

<h1 id="title">Hello World</h1>

<button id="btn">Change Text</button>

<script>38 | P a g e

let heading = document.getElementById("title");

let button = document.getElementById("btn");

button.addEventListener("click", function() {

heading.innerText = "Hello JavaScript!";

heading.style.color = "blue";

});

</script>

Explanation:

• getElementById("title") → Access the <h1> element.

• innerText → Change the text.

• style.color → Change the CSS.

• addEventListener("click", ...) → React to user clicking the button.

Key Points:

• DOM = interface to represent HTML as a tree of objects.

• JavaScript interacts with DOM to dynamically change content, structure, and style.

• Makes webpages interactive.

In short:

• DOM = structured representation of HTML.

• JavaScript = tool to manipulate that structure dynamically.

Question 2: Explain the methods getElementById(), getElementsByClassName(),and querySelector() used to select elements from the DOM.

1. getElementById()

• Selects a single element with the specified id.

• Returns one element object.39 | P a g e

• Fast and widely used.

Syntax:

```
let element = document.getElementById("elementId");
```

Example:

```
<h1 id="title">Hello</h1>
<script>
let heading = document.getElementById("title");
heading.innerText = "Hello JavaScript!";
</script>
```

2. getElementsByClassName()

• Selects all elements with the specified class name.

• Returns an HTMLCollection (like an array, but not exactly).

• Can access elements using index.

Syntax:

```
let elements = document.getElementsByClassName("className");
```

Example:

```
<p class="text">Paragraph 1</p>

<p class="text">Paragraph 2</p>

<script>

let paragraphs = document.getElementsByClassName("text");

paragraphs[0].style.color = "red"; // First paragraph

paragraphs[1].style.color = "blue"; // Second paragraph

</script>
```

3. querySelector()

• Selects the first element that matches a CSS selector.

• Very flexible: can use id (#id), class (.class), tag name, attribute selectors.

Syntax:40 | P a g e

```
let element = document.querySelector("CSS selector");
```

Example:

```
<p class="text">Paragraph 1</p>

<p class="text">Paragraph 2</p>

<script>

let firstParagraph = document.querySelector(".text");

firstParagraph.style.fontWeight = "bold"; // Only first element

</script>
```

Note: To select all matching elements, use querySelectorAll() (returns a NodeList).

Summary Table

Method

Returns

Selects

Notes

getElementById("id")

Single element

Element with

specific id

Fast, only one

element

getElementsByClassName("class") HTMLCollection All elements

with class

Access by index

querySelector("selector")

Single element

First element

matching CSS

selector

Flexible, works

with id, class,

tag

querySelectorAll("selector")

NodeList

All elements

matching CSS

selector

Use for

multiple

elements

In short:

• getElementById → fastest, single element by ID

• getElementsByClassName → multiple elements by class

• querySelector → first element using CSS selector

JavaScript Timing Events (setTimeout, setInterval)41 | P a g e

Question 1: Explain the setTimeout() and setInterval() functions in

JavaScript. Howare they used for timing events?

1. setTimeout()

• Purpose: Executes a function once after a specified delay (in milliseconds).

• Useful for delayed actions like showing messages, animations, or timeouts.

Syntax:

setTimeout(function, delay);

• function → the function to execute

• delay → time in milliseconds (1000 ms = 1 second)

Example:

setTimeout(function() {

alert("Hello after 3 seconds!");

}, 3000); // 3000 ms = 3 seconds

The function runs once after 3 seconds.

2. setInterval()

• Purpose: Executes a function repeatedly at a fixed time interval (in milliseconds).

• Useful for clocks, timers, animations, or polling data.

Syntax:

setInterval(function, interval);

• function → function to execute

• interval → time in milliseconds between executions

Example:

let count = 1;

let intervalId = setInterval(function() {

console.log("Count: " + count);

count++;

if(count > 5) {

clearInterval(intervalId); // stop the interval after 5 times42 | P a g e

}

}, 1000);

Prints Count: 1 to Count: 5 at 1-second intervals.

Key Points

Function

Executes

Use Case

setTimeout() Once

Delayed message, timeout

setInterval()

Repeatedly Clock, timer, repeated actions

• Stopping a timer:

o clearTimeout(timeoutId) → stops a timeout

o clearInterval(intervalId) → stops an interval

In short:

• setTimeout → delay once

• setInterval → repeat at intervals

• Both control timing events in JavaScript.

Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.

```
console.log("Action will happen in 2 seconds...");
setTimeout(function() {
console.log("This message appears after 2 seconds!");
}, 2000);
```

Explanation:

setTimeout() waits for 2000 milliseconds (2 seconds).43 | P a g e

After the delay, it executes the function inside, printing the message.

The first console.log runs immediately; the second runs after 2 seconds.

Output (timeline):

Immediately: Action will happen in 2 seconds...

After 2 seconds: This message appears after 2 seconds!

JavaScript Error Handling

Question 1: What is error handling in JavaScript? Explain the try,

catch, and finally blocks with an example

• Error handling is the process of responding to runtime errors in a program without

stopping the entire execution.

• Helps make programs robust and prevent crashes.

try, catch, and finally

1. try block

o Contains the code that may throw an error.

2. catch block

o Executes if an error occurs in the try block.

o Receives an error object containing details about the error.

3. finally block (optional)

o Executes always, whether an error occurred or not.

o Useful for cleanup actions like closing files, stopping timers, etc.

Syntax:

try {44 | P a g e

// code that may throw an error

} catch(error) {

// code to handle the error

} finally {

// code that always runs

}

Example:

```
try {
let result = riskyOperation(); // may throw an error
console.log("Result:", result);
} catch(error) {
console.log("An error occurred:", error.message);
} finally {
console.log("This block runs always, error or not.");
}
// Example function that throws an error
function riskyOperation() {
throw new Error("Something went wrong!");
}
```

Output:

```
An error occurred: Something went wrong!
This block runs always, error or not.
```

Key Points:

• try → run code safely.

• catch → handle errors gracefully.

• finally → always runs, for cleanup tasks.45 | P a g e

• Helps prevent program crashes and improves reliability.

In short:

Error handling = safely dealing with runtime errors using try-catch-finally.

Question 2: Why is error handling important in JavaScript

applications?

1. Prevents Program Crashes

o Without error handling, a runtime error stops the whole script.

o Proper handling ensures the application keeps running smoothly even if an

error occurs.

2. Improves User Experience

o Users see friendly messages instead of broken pages or console errors.

o Example: Showing “Unable to load data” instead of a blank page.

3. Helps Debugging

o Error objects provide information about the error (message, type, stack

trace).

o Makes it easier for developers to find and fix issues.

4. Maintains Data Integrity

o Ensures critical operations (like saving data, processing forms) are completed

safely.

o Prevents corrupted data or inconsistent application states.

5. Supports Asynchronous Operations

o In modern JS apps using APIs or timers, errors can occur at any time.

o Error handling ensures asynchronous tasks are managed safely.

Example:

```
try {
let data = JSON.parse("Invalid JSON"); // Will throw an error
} catch(error) {46 | P a g e
console.log("Failed to parse data:", error.message);
}
console.log("App continues running...");
```

Output:

```
Failed to parse data: Unexpected token I in JSON at position 0
App continues running...
```

In short:

• Error handling ensures your JavaScript application is robust, user-friendly, and

reliable.

• Without it, errors can crash apps, frustrate users, and cause data loss.